Sicherheitsprüfung – SPS Monitor

HTML-Oberfläche und lokale Brücke (SpsBruecke) zum Lesen und Schreiben von DB-Werten einer Siemens S7-315 über Ethernet
Fassung der Software nach Härtung: **1.5** · Prüfdatum: **15.10.2026** · Prüfung im Auftrag des Betreibers

1. Zusammenfassung

Die HTML-Oberfläche und die zugehörige Brücke wurden nach denselben Maßstäben geprüft wie das Schwesterwerkzeug (SuR-Tool): Code-Durchsicht gegen übliche IT-Sicherheitsvorgaben, regelbasierte Suche nach Schwachstellenmustern, eine Lieferkettenprüfung der verwendeten Fremdbausteine sowie eine automatisierte Abnahme im echten Browser.

Die Erstfassung wies **mehrere gravierende Schwächen** auf. Die schwerste: Die Brücke nahm Schreibbefehle von *jeder* Webseite im selben Browser an (offene CORS-Freigabe, keine Herkunftsprüfung) – eine beliebige Internetseite hätte, während die Brücke läuft, ungefragt in die laufende Anlage schreiben können. Dazu kamen zwei Wege zum Einschleusen von Skript (XSS), ein irreführender Schreib-Bestätigungsdialog, fehlende Eingabeprüfung und das Fehlen einer Inhaltssicherheits-Richtlinie (CSP).

> **Ergebnis nach Härtung:** Alle als Kritisch, Hoch und Mittel eingestuften Befunde sind **behoben** und durch automatisierte Prüfungen abgesichert (31 Prüfpunkte, alle bestanden). Ein verbleibender Punkt (Start der Brücke über eine eigene Adressart) ist mit Begründung als vertretbar bewertet. Die verwendete Fremdbibliothek `Sharp7` ist das **echte, unbedenkliche** Paket – *nicht* das Schadpaket `Sharp7Extend`.

2. Prüfgegenstand

Bestandteil	Beschreibung
`oberflaeche.html`	Einzelne HTML-Datei, Bedienoberfläche im Browser (Edge/Chrome), ohne externe Bibliotheken
`Program.cs`	Die Brücke: handgeschriebener Mini-Webserver, nimmt die Oberfläche an und spricht per S7 mit der Steuerung
`PlcService.cs`	S7-Kommunikation über die Bibliothek Sharp7
`SPS Monitor.hta`, `*.bat`, `Installieren.ps1`	Start- und Bauwege unter Windows (ohne Administratorrechte)
`SpsBruecke.csproj`	Projekt- und Abhängigkeitsdefinition (NuGet)

3. Vorgehen und Methodik

- **Code-Durchsicht** beider Seiten (Browser-JavaScript und C#) mit besonderem Blick auf Schreibpfade zur Steuerung, da Fehlbedienung reale Anlagentechnik schalten kann.
- **Regelbasierte Mustersuche** nach gefährlichen Konstrukten (HTML-Senken wie `innerHTML`, `eval`; ungeprüfte Eingaben; fehlende Herkunfts- und Größenprüfungen).
- **Automatisierte Abnahme im echten Browser** (Chromium über Playwright): die gehärtete Oberfläche wurde gegen eine originalgetreue Nachbildung der gehärteten Brücke geführt und jeder Schutzmechanismus einzeln nachgewiesen.
- **Lieferkettenprüfung** der NuGet-Abhängigkeit anhand aktueller Hersteller- und Sicherheitshinweise.

Hinweis zum Umfang: Die Prüfung lief in einer abgeschotteten Bauumgebung ohne Zugang zu externen Paketquellen (PyPI/npm). Etablierte Scanner (Semgrep, npm audit) waren dort nicht installierbar; sie wurden durch die oben genannten eigenen Regelsätze und die Browser-Abnahme ersetzt. Für einen regelmäßigen Betrieb wird empfohlen, zusätzlich einen Scanner in die Baukette aufzunehmen (siehe Abschnitt 8).

4. Bewertungsmaßstab

Eingestuft wird nach Schwere (Kritisch/Hoch/Mittel/Niedrig/Hinweis), orientiert an OWASP Top 10 und OWASP ASVS sowie der jeweiligen CWE-Kennung. Maßgeblich ist die mögliche Auswirkung auf die **Anlage** – also ob ein Befund zu ungewolltem Schreiben, zu falscher Anzeige oder zum Ausfall der Bedienung führen kann.

5. Befunde und Behebung

Nr.	Befund	Schwere	CWE	Status
B1	Brücke nimmt Schreibbefehle jeder Webseite an (offenes CORS, keine Herkunfts-/Rechteprüfung)	Kritisch	306, 346, 352	behoben
B2	Schreib-Rückfrage irreführend: angezeigter Name und tatsächliche DB-Adresse entkoppelt	Kritisch	451	behoben
B3	Skript-Einschleusung (XSS) über das Feld `version` einer Brückenantwort	Hoch	79	behoben
B4	Skript-Einschleusung (XSS) über die eingegebene Brückenadresse	Hoch	79	behoben
B5	Datenabfluss: Brückenadresse unbegrenzt umlenkbar, Projektübernahme ohne Prüfung	Hoch	200, 942	behoben
B6	Keine Inhaltssicherheits-Richtlinie (CSP), keine Trusted Types	Hoch	693, 1021	behoben
B7	Geladene Projekte ohne Wertebereichs-, Typ- und Mengenprüfung	Mittel	20	behoben
B8	Unbegrenzte Anfragegröße in der Brücke (`Content-Length`) – Speichererschöpfung	Mittel	400, 789	behoben

B9	SPS-Aufrufe ohne Zeitgrenze/Antwortprüfung, Auto-Lesen ohne Wiedereintrittssperre	Mittel	400	**behoben**
B10	CSV-Import verwirft Aufzeichnung ohne Rückfrage, legt scheinbar beschreibbare Tags an, keine Mengengrenze	Mittel	20, 451	**behoben**
B11	Verwechselbare Tag-Namen (`toString` u. a.) täuschen Messwerte vor	Niedrig	1321	**behoben**
B12	Fehlende Sicherheits-Kopffelder der Brücke (nosniff, Frame-Schutz)	Niedrig	693	**behoben**
B13	Eigene Adressart `spsmonitor://` erlaubt jeder Seite den *Start* der Brücke	Niedrig	—	**akzeptiert***

* Siehe Abschnitt 7, letzter Punkt: durch die Token-Pflicht ist ein Start ohne anschließende Benutzung wirkungslos; der Browser fragt zudem vor jedem Protokollaufruf nach.

Die wichtigsten Befunde im Einzelnen

B1 – Brücke nahm Befehle jeder Webseite an (Kritisch)

Datei: `Program.cs` (CORS-Kopf, Routen)

Die Brücke sendete auf jede Antwort `Access-Control-Allow-Origin: *` und prüfte bei den schreibenden Routen weder Herkunft noch Berechtigung. Solange die Brücke lief, hätte eine beliebige im selben Browser geöffnete Webseite (Intranet, Lieferantenportal, Werbe- oder Phishing-Seite) ungefragt `/api/verbinden` und `/api/schreiben` aufrufen können – ein Schreibzugriff auf die laufende Anlage ohne jede Rückfrage.

> **Behebung:** Die offene CORS-Freigabe wurde entfernt. Zusätzlich vergibt die Brücke bei jedem Start ein zufälliges **Sitzungs-Token**, das nur in die von ihr selbst ausgelieferte Seite eingesetzt wird. Alle steuernden Routen (`verbinden`, `schreiben`, `trennen`, `beenden`, `projekt`-Speichern) verlangen dieses Token in einem eigenen Kopffeld. Eine fremde Seite kennt es nicht, und das eigene Kopffeld erzwingt eine Vorabfrage, die ohne Freigabe scheitert.

B2 – Irreführende Schreib-Rückfrage (Kritisch)

Datei: `oberflaeche.html`, Funktion `tagSchreiben`

Die Rückfrage vor dem Schreiben nannte den *Namen* führend; Name, Beschriftung und sogar die DB-Adresse stammen aber alle aus der Projektdatei. Ein präpariertes Projekt konnte so einen harmlos benannten Schalter zeigen, der in Wahrheit eine ganz andere, sicherheitsrelevante Adresse beschreibt.

> **Behebung:** Die Rückfrage zeigt jetzt die **technische Zieladresse zuerst und deutlich** (z. B. `DB14.DBX212.3`), darunter erst Name und Auslöser. Alle eingesetzten Texte werden gekürzt und von Zeilenumbrüchen befreit, damit der Warnhinweis nicht aus dem Dialog geschoben werden kann.

B3/B4 – Zwei Wege zur Skript-Einschleusung (Hoch)

Datei: `oberflaeche.html`, Brückenstatus-Anzeige

Das Feld `version` aus der Brückenantwort und die eingegebene Brückenadresse wurden ungeprüft als HTML eingesetzt. Über eine untergeschobene Antwort oder eine präparierte Adresse ließ sich Skript im Ursprung der Seite ausführen – und damit `tagSchreiben` samt Rückfrage umgehen.

> **Behebung:** Beide Werte werden jetzt ausschließlich als Text gesetzt (keine HTML-Auswertung mehr). Die Adresse wird über `new URL(...).origin` auf reines http/https mit Host und Port geprüft; Sonderzeichen können nicht mehr durchkommen. Zusätzlich greift die neue CSP (B6) als zweite Sicherung.

B6 – Keine CSP / keine Trusted Types (Hoch)

Datei: `oberflaeche.html` (Kopf), `bauen.py`

Anders als das SuR-Tool besaß die Seite keine Inhaltssicherheits-Richtlinie – den Einschleus-Wegen fehlte jede zweite Verteidigungslinie.

> **Behebung:** Eine strenge CSP wurde eingezogen: nur das eine eingebaute Skript darf laufen (per SHA-256-Hash, von `bauen.py` automatisch eingesetzt), keine externen Quellen, Verbindungen nur zur eigenen Brücke (`connect-src` auf `self` /localhost – das unterbindet zugleich den Datenabfluss aus B5), und `require-trusted-types-for 'script'` erzwingt, dass HTML nur über eine geprüfte Richtlinie ins Dokument gelangt.

B7/B8/B9/B10 – Eingabe-, Größen- und Ablaufprüfungen (Mittel)

- **B7:** Geladene Projekte werden jetzt vollständig geprüft: Wertebereiche geklemmt (DB 0–65535, Bit 0–7, Positionen/Größen auf die Leinwand), Typen und Symbolnamen gegen Weißlisten, Mengen begrenzt (max. 500 Tags, 50 Seiten, 300 Elemente, Texte 200 Zeichen). Ein fehlerhaftes Element bricht den Aufbau nicht mehr ab.
- **B8:** Die Brücke weist Anfragen über 4 MB mit `413` ab, statt Speicher in Höhe der angegebenen Länge zu belegen.
- **B9:** Alle SPS-Aufrufe laufen über eine gemeinsame Funktion mit Zeitgrenze und Prüfung auf Erfolg und JSON-Antwort; das Auto-Lesen hat eine Wiedereintrittssperre.
- **B10:** Der CSV-Import fragt vor dem Verwerfen einer laufenden Aufzeichnung nach, begrenzt Spalten/Zeilen und legt CSV-Spalten als reine **Anzeigekurven ohne SPS-Adresse** an – für sie gibt es kein Schreibfeld.

6. Lieferkette: Sharp7 ist unbedenklich, Sharp7Extend ist Schadsoftware

> Im November 2025 wurde ein **Schadpaket namens** `Sharp7Extend` öffentlich – eine Tippfehler-Fälschung der echten Bibliothek `Sharp7`. Es bündelt die echte Bibliothek mit Schadcode, der nach einer zufälligen Verzögerung Schreibzugriffe auf S7-Steuerungen **heimlich scheitern** lässt (gemeldete Erfolgsmeldungen trotz nicht geschriebener Werte) und zufällig Verbindungen abbricht – gezielt gegen Industriesteuerungen.

> **Dieses Projekt ist nicht betroffen.** Eingebunden ist ausschließlich das echte Paket `Sharp7` von `fede.barresi` (`github.com/fbarresi/Sharp7`), fest auf Version **1.1.84**. Das Schadpaket `Sharp7Extend` kommt an keiner Stelle vor.

Zusätzliche Absicherung (umgesetzt): In der Projektdatei sind jetzt die NuGet-Prüfung (`NuGetAudit`, meldet bekannte Schwachstellen beim Bauen) und eine Sperrdatei (`packages.lock.json` mit exakten Versionen und Prüfsummen) aktiviert; die genaue Paketidentität ist dort dokumentiert, damit künftige Bauläufe nicht versehentlich ein Fälschungspaket ziehen.

7. Umgesetzte Härtung – Überblick

Bereich	Maßnahme
Zugriffsschutz Brücke	Offenes CORS entfernt; Sitzungs-Token für alle steuernden Routen; Herkunfts-/Loopback-Prüfung
Robustheit Brücke	Anfragegröße begrenzt (413); Sicherheits-Kopffelder (nosniff, Frame-Schutz, Referrer)
Browser-Schutz	Strenge CSP mit Skript-Hash; Trusted Types; nur Verbindung zur eigenen Brücke
Skript-Einschleusung	Gefährliche HTML-Senken auf reinen Text umgestellt; Adressprüfung über `URL`
Schreibsicherheit	Rückfrage mit technischer Adresse führend; Längen gekürzt; Anzeigekurven nicht beschreibbar
Eingabeprüfung	Wertebereiche, Typ-/Symbol-Weißlisten, Mengengrenzen, abgesicherter Aufbau
Lieferkette	Paket gepinnt und dokumentiert; NuGet-Audit und Sperrdatei aktiviert

8. Restrisiken und Empfehlungen

- **Keine Benutzeranmeldung (bauartbedingt):** Wer die von der Brücke ausgelieferte Oberfläche im Netz erreicht, darf die Anlage bedienen. Das Token schützt gegen fremde Webseiten und lose Dateien, nicht gegen berechtigte Geräte im selben Netz. **Empfehlung:** Brücke nur an `localhost` binden, wo kein Netzzugriff gewünscht ist; sonst Zugang über Netzsegmentierung/Firewall mit der IT abstimmen (wie bereits in der Anleitung vermerkt).
- **Nur-HTTP im Netzbetrieb:** Der Verkehr zwischen Gerät und Brücke ist unverschlüsselt. Für reine `localhost`-Nutzung unerheblich; bei Netzbetrieb über ein vertrauenswürdiges Segment führen.
- **Protokoll-Start (B13):** Der `spsmonitor://`-Behandler bleibt der Bequemlichkeit halber bestehen; durch die Token-Pflicht ist ein von außen ausgelöster Start ohne Nutzen. Wer das ausschließen will, startet die Brücke nur über `SPS Monitor starten.bat`.
- **Scanner in der Baukette:** Für den laufenden Betrieb empfiehlt sich, in einer Umgebung mit Netzzugang einmalig `npm audit`/Semgrep (Browser-Teil) sowie `dotnet list package --vulnerable` (Brücke) laufen zu lassen und die erzeugte `packages.lock.json` mitzuführen.

9. Abnahmeprüfungen (automatisiert bestanden)

Die gehärtete Oberfläche wurde im echten Browser (Chromium) gegen eine Nachbildung der gehärteten Brücke geführt. Bestanden wurden u. a.:

- Seite lädt unter strenger CSP und Trusted Types, Token wird gelesen, Skript-Hash passt.
- Verbinden/Lesen/Schreiben funktionieren mit Token; Schreiben **ohne** Token wird mit `403` abgewiesen.
- Die Schreib-Rückfrage nennt die technische Adresse (`DB…`) führend samt Warnhinweis.
- Ein untergeschobenes `version`-Feld und eine präparierte Adresse führen **kein** Skript aus (als Text dargestellt, kein `<img>` erzeugt).
- Ein Projekt mit unsinnigen Werten wird vollständig geklemmt (Mengen, Bereiche, Typen).
- CSV-Spalten werden Anzeigekurven ohne Schreibfeld; eine laufende Aufzeichnung wird nur nach Rückfrage ersetzt; der Projekt-Roundtrip erhält den Anzeige-Status.
- Bestehende Funktionen unverändert: Symbol anlegen, Projekt speichern (gemeinsames `projekt.json`), CSV-Export, Auto-Aktualisierung, Brücke stoppen.

10. Anhang

Geänderte Dateien

`Program.cs · oberflaeche-vorlage.html · bauen.py · SpsBruecke.csproj · SPS Monitor.hta · .gitattributes (neu)`

Inhaltssicherheits-Richtlinie (Wortlaut)

```
default-src 'none'; script-src 'sha256-…'; style-src 'unsafe-inline'; img-src data: blob:; connect-src 'self' http://localhost:* http://127.0.0.1:*; base-uri 'none'; form-action 'none'; require-trusted-types-for 'script'; trusted-types spsmonitor
```

Erstellt im Rahmen der Weiterentwicklung des SPS Monitor. Diese Prüfung ersetzt keine förmliche Zertifizierung; sie dokumentiert die vorgenommene Härtung gegen übliche IT-Sicherheitsvorgaben und die durchgeführten Nachweise. Quellen zur Lieferkette: Socket (socket.dev), Industrial Cyber, NuGet.org – Stand Oktober 2026.